



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Module Objective: In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```

CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set                // To store unique string fact
    DECLARE rules AS List               // To store rules as Tuples: (

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules
  
```

```
FUNCTION clear_facts():  
    EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():  
    DECLARE new_facts_added = TRUE  
  
    WHILE new_facts_added == TRUE:  
        new_facts_added = FALSE  
  
        FOR EACH (premises, conclusion) IN rules:  
            IF conclusion IS NOT IN facts:  
  
                // Modus Ponens Check  
                IF ALL premises ARE IN facts:  
                    ADD conclusion TO facts  
                    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py`, instantiate the KB in the `__init__` method: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
Rule 1: `TargetVisible  $\wedge$  HasDust  $\Rightarrow$  SafeToEngage`
Rule 2: `SafeToEngage  $\wedge$  BloodseekerMissing  $\Rightarrow$  Retreat`

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list`, clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()`.
4. Run `self.kb.forward_chain()`.
5. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```

from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce Sa
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()

```

Part 5: Theory-to-Implementation Mapping

Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your

new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

- **Reachability (Physical Constraints):** Checked via spatial boundaries ($0 \leq nx < \text{width}$ and $0 \leq ny < \text{height}$) and static obstacles (`neighbor_pos` not in `walls`). It determines if the agent physically has a valid geometric path to step onto a grid tile.
- **Feasibility (Logical Constraints):** Checked via high-level rules stored in the Knowledge Base. A tile can be physically reachable yet logically infeasible if standing on it violates safety or game constraints.
- **Enforcement by Knowledge Base:** Prior to adding a reachable neighbor to the A* priority queue, the agent resets transient KB facts using `clear_facts()` , populates the facts specific to that neighbor's percepts (`tell_fact()`), and runs `forward_chain()` . If logical deduction yields an unsafe outcome (such as 'Retreat' in `self.kb.facts`), the tile is flagged as infeasible and pruned via `continue` , preventing A* from building paths through dangerous states.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

- **Separation of Knowledge and Control:** A declarative approach decouples the reasoning algorithm (`forward_chain()`) from the domain rules (facts and Horn clauses). Procedural code mixes business logic directly into the control flow, requiring developers to manually manage evaluation order and execution paths.
- **Handling Multi-Step Inferences:** Procedural `if` statements fail to automatically handle transitive derivations (e.g., Fact A implies Fact B, and Fact B implies Fact C). To achieve this procedurally, code must be written in a specific order or explicitly re-evaluated across multiple nested conditionals. The forward chaining algorithm handles arbitrary multi-step chaining automatically until a fixed point is reached.
- **Impact of Adding 50 Rules:**
 - **Procedural Approach:** Results in exponential conditional nesting ("spaghetti code"), making updates error-prone, hard to trace, and difficult to test.
 - **Declarative Approach:** Code complexity remains constant. Adding 50 rules requires only 50 distinct `kb.tell_rule(...)` registrations. The inference loop executes unchanged regardless of rule count.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

- **Formal Definition:** A Horn clause has the form $P^1 \text{ and } P^2 \text{ and } \dots \text{ and } P^n \text{ implies } Q$, where a conjunction of positive premises implies a single positive conclusion.
- **Execution of Modus Ponens:** The classical Modus Ponens inference rule states that if $P \text{ implies } Q$ is true and P is known to be true, then Q must be true. In code, this rule is executed by checking:

```
if all(premise in self.facts for premise in
premises): self.facts.add(conclusion)
```
- This directly checks if the complete antecedent ($P^1 \text{ and } P^2 \dots$) exists in `self.facts`. If satisfied, Modus Ponens fires and asserts Q into `self.facts`.
- **The Iterative Loop as Fixed-Point Deduction:** The `while new_facts_added:` loop continuously applies Modus Ponens across all registered rules. It iterates until a full pass generates no new facts, mathematically ensuring the KB reaches logical closure (all possible entailments under the given percepts have been derived).

Finalize and Lock Lab 05 Implementation

✅ **Lab 05 Theory-to-Implementation Mapping completed!**



FACULTY OF COMPUTING